COMPTE RENDU TECHNIQUE

Projet Ansible — Déploiement automatisé de serveurs Minecraft sur Proxmox LXC

Sommaire

1. Vue d'ensemble du projet
2. Architecture générale
3. Fichiers de configuration
   3.1 ansible.cfg
   3.2 inventory.ini
   3.3 ct_registry.csv
4. Le Playbook deploy.yml
   4.1 Play 1 — Calcul ID et IP
   4.2 Play 2 — Création du conteneur LXC
   4.3 Play 3 — Installation de Minecraft
5. Flux d'exécution complet
6. Paramètres et variables
7. Points forts et limitations
8. Recommandations d'amélioration

1. Vue d'ensemble du projet

Ce projet met en place une chaîne d'automatisation complète permettant de déployer un nouveau serveur Minecraft en une seule commande Ansible. L'objectif est de provisionner dynamiquement des conteneurs LXC sur un nœud Proxmox, d'y installer Java 21 et le serveur Minecraft, puis de démarrer le service automatiquement.

Chaque exécution du Playbook deploy.yml crée un nouveau conteneur isolé, avec un identifiant (CTID) et une adresse IP uniques, calculés automatiquement grâce à un registre ct_registre.csv persistant.

Élément	Valeur
Nœud de contrôle	VM Debian avec Ansible
Hyperviseur cible	Proxmox — nœud pve-02 (10.2.0.253)
Type de virtualisation	Conteneurs LXC
OS des conteneurs	Debian 12 (Template debian-12-standard)
Jeu déployé	Minecraft Java Edition (port 25565)
Runtime Java	JDK 21 (oracle)
Gestion des identifiants	Registre CSV local (ct_registry.csv)

2. Architecture générale

L'architecture repose sur trois couches distinctes qui communiquent successivement :

Couche 1 — Nœud de contrôle Ansible (VM Debian)

C'est depuis cette machine que toutes les opérations sont initiées. Elle héberge les fichiers de configuration Ansible, le Playbook deploy.yml, l'inventaire, et le registre CSV des conteneurs. Ansible n'est pas installé sur les machines cibles : il se connecte à elles via SSH (ou via l'API REST de Proxmox).

Couche 2 — Proxmox (hyperviseur)

Proxmox expose une API REST sur le port 8006. Ansible s'authentifie auprès de cette API pour créer les conteneurs LXC. Le nœud pve-02 possède le Template Debian 12 en local (stockage local) et héberge les conteneurs créés.

Couche 3 — Conteneurs LXC (serveurs Minecraft)

Chaque conteneur est un environnement Debian 12 isolé, avec 2 Go de RAM, 2 vCPUs et 8 Go de stockage. Une fois créé et démarré par Proxmox, Ansible s'y connecte en SSH pour y installer Java 21 et le serveur Minecraft, puis configure un service systemd pour le maintenir actif.

Schéma des communications :

VM Debian / Ansible	→	Proxmox API REST :8006	→	LXC Minecraft :25565

Schéma de l'infrastructure :

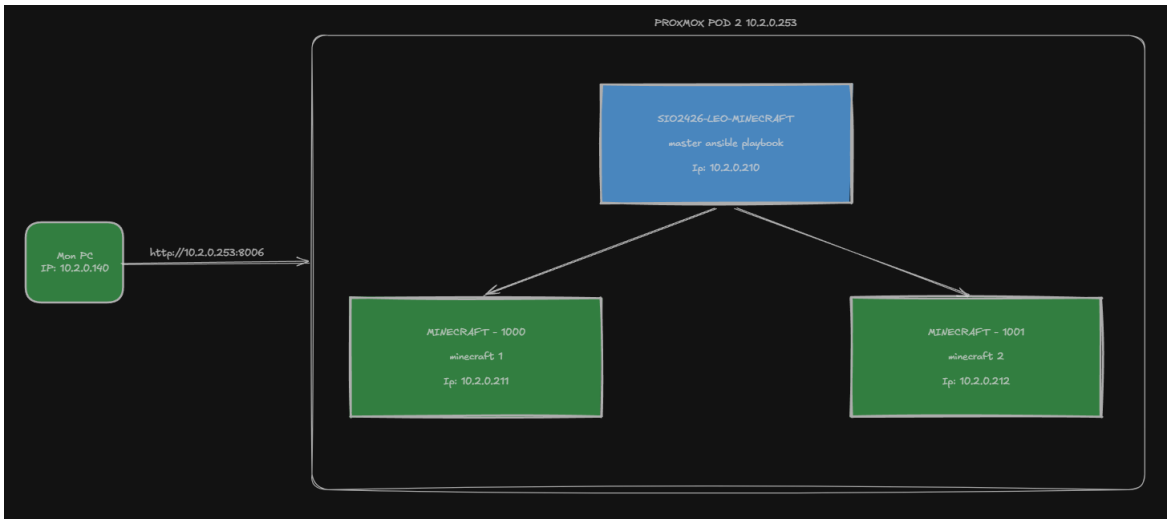
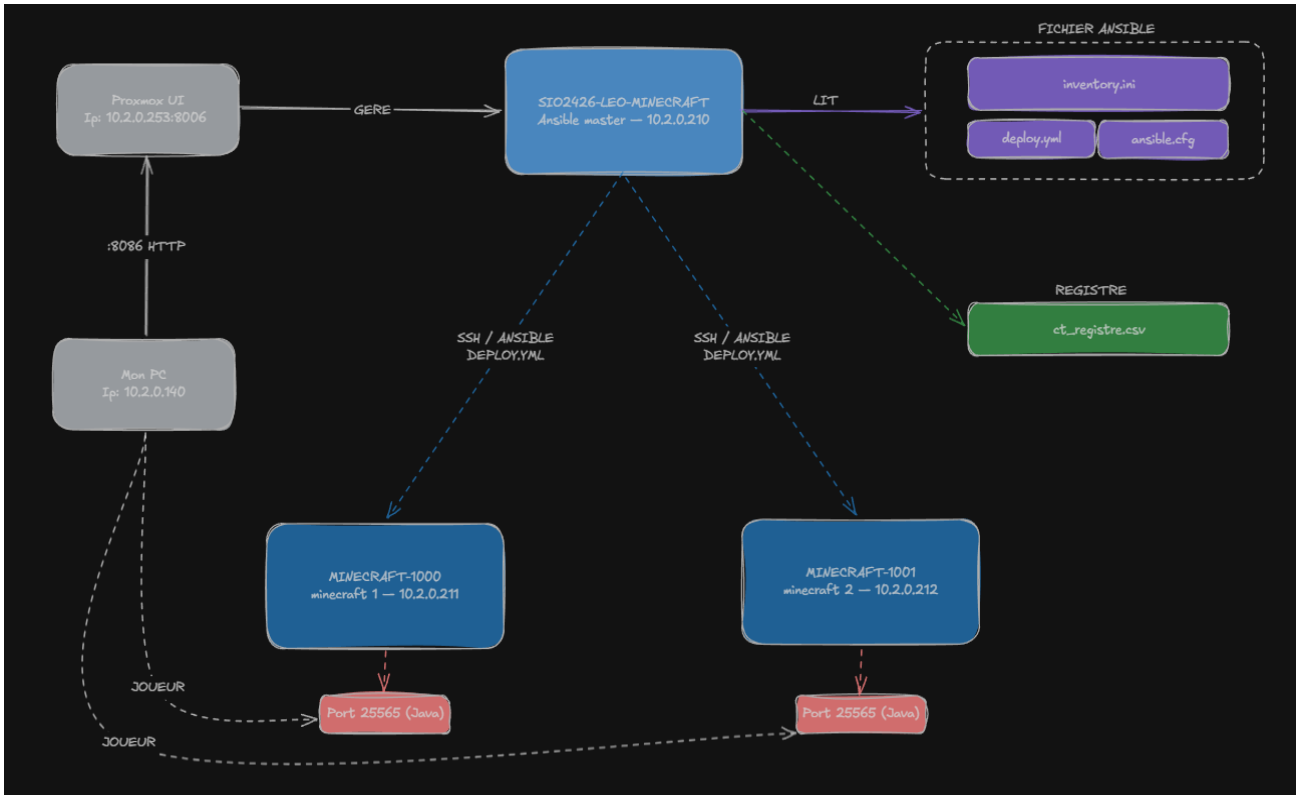
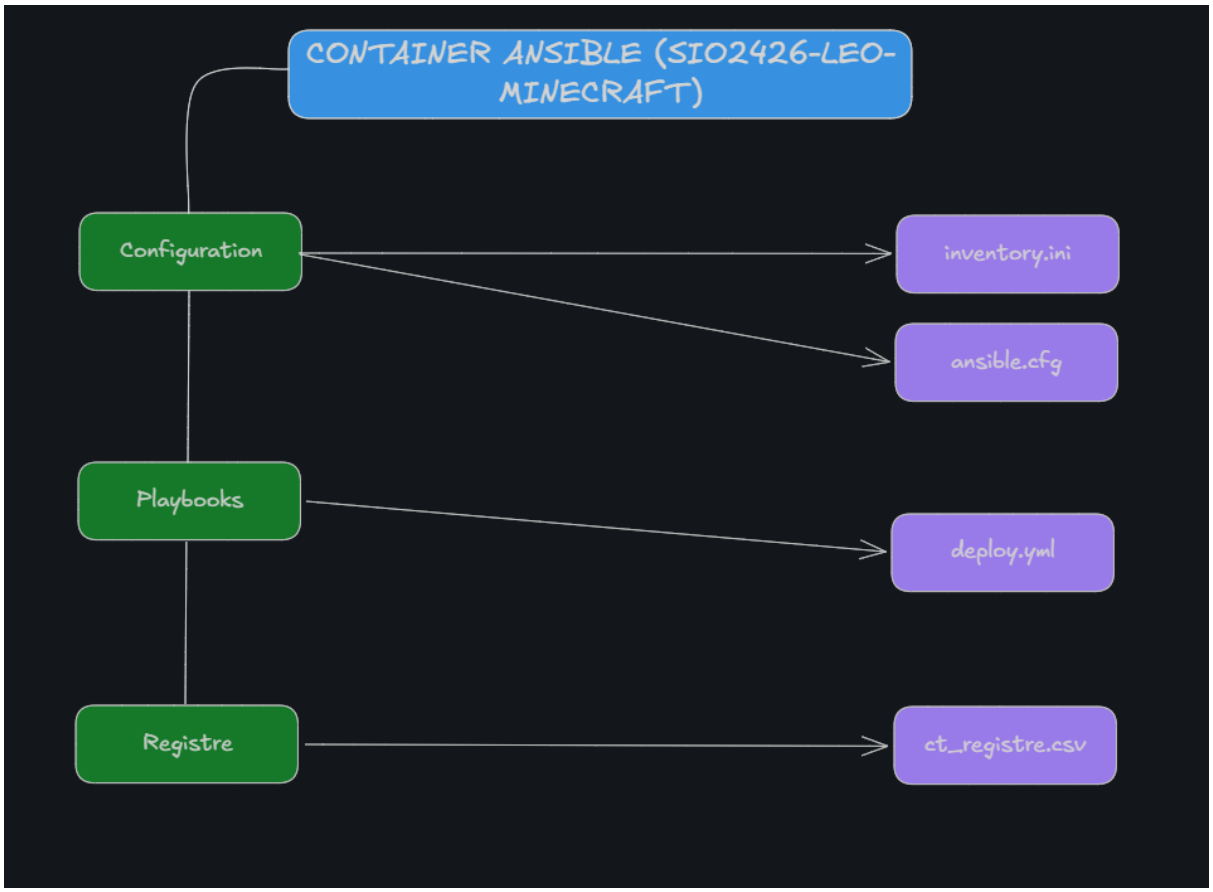


Schéma des flux :



3. Fichiers de configuration



3.1 ansible.cfg

```
[all:vars]
ansible_python_interpreter=/usr/bin/python3
root@debian:~/ansible# cat ansible.cfg
[defaults]
inventory           = ./inventory.ini
host_key_checking   = False
retry_files_enabled = False
stdout_callback     = ansible.builtin.default
result_format       = yaml
timeout             = 60
deprecation_warnings = False

[ssh_connection]
ssh_args    = -o ControlMaster=auto -o ControlPersist=60s -o StrictHostKeyChecking=no
pipelining  = True
```

Ce fichier centralise tous les paramètres globaux du nœud de contrôle Ansible. Il est divisé en deux sections principales.

Section [defaults]

Paramètre	Rôle et impact
inventory = ./inventory.ini	Chemin vers l'inventaire statique des hôtes connus.
host_key_checking = False	Désactive la vérification SSH des empreintes. Indispensable pour les conteneurs créés dynamiquement dont la clé n'est pas encore connue.
retry_files_enabled = False	Supprime la création de fichiers .retry après un échec. Garde le répertoire propre.
stdout_callback = default	Format de sortie console standard Ansible.
result_format = yaml	Affiche les résultats des tâches en YAML, plus lisible que le JSON compact.
timeout = 60	Délai maximal (secondes) pour les connexions SSH.
deprecation_warnings = False	Masque les avertissements de fonctionnalités dépréciées pour une sortie plus claire.

Section [ssh_connection]

Paramètre	Rôle et impact
ControlMaster=auto	Active le multiplexage SSH : une seule connexion TCP est maintenue par hôte, toutes les tâches la réutilisent. Gain de temps significatif.
ControlPersist=60s	La connexion maître reste active 60 secondes après la dernière tâche, prête à être réutilisée.
StrictHostKeyChecking=no	Accepte automatiquement les nouvelles clés SSH sans confirmation interactive.
pipelining = True	Réduit les aller-retours SSH en envoyant les modules Python directement dans le canal, sans créer de fichier temporaire sur l'hôte distant.

3.2 inventory.ini

```
[all:vars]
ansible_python_interpreter=/usr/bin/python3
```

L'inventaire statique déclare uniquement la machine locale (localhost) avec la variable globale ansible_python_interpreter=/usr/bin/python3, qui force l'utilisation de Python 3 pour l'exécution des modules.

Point important : les conteneurs Minecraft NE ne figurent PAS dans cet inventaire. Ils y sont ajoutés dynamiquement en cours d'exécution via le module add_host du Playbook. C'est ce mécanisme qui permet à Ansible de se connecter à des machines qu'il ne connaissait pas au démarrage.

3.3 ct_registry.csv

Ce fichier CSV joue le rôle de base de données légère et persistante. Il est au cœur du mécanisme d'attribution des identifiants uniques.

Structure de chaque ligne :

Colonne	Exemple	Description
CTID	1000	Identifiant unique du conteneur LXC dans Proxmox
IP	10.2.0.211	Adresse IP attribuée au conteneur
Horodatage	2025-06-01T14:32:05+02:00	Date et heure ISO 8601 de création

Fonctionnement : le Playbook compte le nombre de lignes existantes dans ce fichier pour en déduire les prochains CTID et IP disponibles. Si le fichier est vide ou inexistant, le premier conteneur obtient le CTID 1000 et l'IP 10.2.0.211. Le deuxième obtiendra 1001 et 10.2.0.212, et ainsi de suite.

Attention : ce mécanisme séquentiel ne gère pas la suppression de conteneurs. Si un conteneur est supprimé de Proxmox mais que sa ligne reste dans le CSV, le CTID et l'IP ne seront pas réutilisés.

4. Le Playbook deploy.yml

Le Playbook est structuré en trois plays successifs. Chaque Play a une responsabilité bien définie et s'exécute sur un groupe d'hôtes différent.

4.1 Play 1 — Calcul de l'ID et de l'IP

```yaml
---
- name: "Calcul IP et ID"
  hosts: localhost
  gather_facts: false
  vars:
    registry_file: "./ct_registry.csv"
    ip_base: "10.2.0"
    ip_start: 211
    id_start: 1000
  tasks:
    - name: Créer le registre si absent
      copy:
        dest: "{{ registry_file }}"
        content: ""
        force: false

    - name: Compter les containers existants
      shell: "grep -c '^[0-9]' {{ registry_file }} || echo 0"
      register: ct_count_raw
      changed_when: false

    - name: Calculer ID et IP
      set_fact:
        ct_id: "{{ id_start | int + ct_count_raw.stdout | trim | int }}"
        ct_ip: "{{ ip_base }}.{{ ip_start | int + ct_count_raw.stdout | trim | int }}"

    - name: Infos container
      debug:
        msg: "CTID={{ ct_id }} | IP={{ ct_ip }}"

    - name: Sauvegarder dans le registre
      lineinfile:
        path: "{{ registry_file }}"
        line: "{{ ct_id }},{{ ct_ip }},{{ lookup('pipe', 'date -Iseconds') }}"

```

Hôte cible : localhost (la VM Ansible elle-même)

Ce Play prépare les variables ct_id et ct_id qui seront utilisées dans les plays suivants. Il s'exécute localement et ne nécessite aucune connexion réseau.

Tâche 1 — Créer le registre si absent

Le module copy avec force: false crée le fichier CSV s'il n'existe pas, sans jamais l'écraser s'il existe déjà. C'est une opération idempotente : elle peut être répétée sans effet secondaire.

Tâche 2 — Compter les conteneurs existants

La commande Shell grep -c '^[0-9]' compte les lignes commençant par un chiffre. Le || echo 0 gère proprement le cas où le fichier est vide (grep retournerait un code d'erreur 1, ce qui ferait échouer la tâche sans ce mécanisme).

```
grep -c '^[0-9]' ./ct_registry.csv || echo 0
```

Tâche 3 — Calculer ID et IP avec set_fact

Le module set_fact effectue deux calculs arithmétiques simples :

Variable	Formule de calcul
ct_id	1000 + nombre_de_lignes_CSV
ct_ip	10.2.0.(211 + nombre_de_lignes_CSV)

Tâche 4 — Sauvegarder dans le registre

Le module lineinfile ajoute immédiatement la nouvelle entrée dans le CSV avec un horodatage généré par lookup('pipe', 'date -Iseconds'). La ligne est écrite avant même que le conteneur soit créé, ce qui garantit la réservation de l'ID même en cas d'échec ultérieur.

4.2 Play 2 — Création du conteneur LXC sur Proxmox

```yaml
    - name: Créer le container LXC
      uri:
        url: "https://{{ proxmox_host }}:8006/api2/json/nodes/{{ proxmox_node }}/lxc"
        method: POST
        headers:
          Cookie: "PVEAuthCookie={{ pve_ticket }}"
          CSRFPreventionToken: "{{ pve_csrf }}"
        body_format: form-urlencoded
        body:
          vmid: "{{ ct_id }}"
          hostname: "minecraft-{{ ct_id }}"
          ostemplate: "local:vztmpl/debian-12-standard_12.12-1_amd64.tar.zst"
          storage: "local"
          rootfs: "local:8"
          memory: 2048
          cores: 2
          net0: "name=eth0,bridge=vmbr0,ip={{ ct_ip }}/24,gw={{ gateway }}"
          nameserver: "8.8.8.8"
          searchdomain: "local"
          ssh-public-keys: "{{ ssh_key }}"
          unprivileged: 1
          start: 1
        validate_certs: false

    - name: Attendre que SSH soit disponible
      wait_for:
        host: "{{ ct_ip }}"
        port: 22
        timeout: 60
        delay: 5

    - name: Ajouter le container à l'inventaire
      add_host:
        name: "{{ ct_ip }}"
        groups: minecraft_cts
        ansible_user: root
        ansible_ssh_private_key_file: "~/.ssh/id_rsa"
        ansible_ssh_extra_args: "-o StrictHostKeyChecking=no"
        mc_ct_id: "{{ ct_id }}"

```

Hôte cible : localhost (communication via API REST Proxmox)

Ce Play interagit avec l'API Proxmox pour créer et démarrer le conteneur. Il utilise le module Uri d'Ansible pour effectuer des requêtes HTTP.

Tâche 1 — Authentification Proxmox

Un POST vers /api2/json/access/ticket avec les credentials root@pam retourne deux tokens de sécurité :

- PVEAuthCookie : ticket de session à envoyer dans le header Cookie
- CSRFPreventionToken : token anti-CSRF à envoyer dans un header dédié

Ces deux tokens sont nécessaires pour toutes les opérations qui modifient l'état de Proxmox (création, démarrage, suppression de VM/CT).

Tâche 2 — Création du conteneur LXC

Un POST vers /api2/json/nodes/pve-02/lxc crée le conteneur avec les paramètres suivants :

Paramètre	Valeur	Description
vmid	ct_id (calculé)	Identifiant unique dans Proxmox
hostname	minecraft-{ct_id}	Nom du conteneur
ostemplate	debian-12-standard_12.12-1	Template OS utilisé
memory	2048 Mo	RAM allouée
cores	2	Nombre de vCPUs
rootfs	local:8	Disque de 8 Go sur stockage local
net0	eth0, vmbr0, IP/24, GW	Interface réseau avec IP statique
nameserver	8.8.8.8	DNS Google
ssh-public-keys	Clé publique RSA	Clé SSH depuis ~/.ssh/id_rsa.pub
unprivileged	1	Conteneur non privilégié (sécurité renforcée)
start	1	Démarre le conteneur automatiquement après création

Tâche 3 — Attente SSH

Le module wait_for attend que le port 22 soit accessible sur l'IP du nouveau conteneur, avec un délai initial de 5 secondes et un timeout de 60 secondes. C'est une barrière de synchronisation essentielle : sans elle, Ansible essaierait de se connecter au conteneur avant qu'il soit prêt.

Tâche 4 — Ajout dynamique à l'inventaire

Le module add_host enregistre le nouveau conteneur dans le groupe minecraft_cts avec tous les paramètres SSH nécessaires. C'est grâce à ce mécanisme que le Play 3 peut cibler un hôte qui n'existait pas au moment du lancement du Playbook.

4.3 Play 3 — Installation de Minecraft

```yaml
- name: "Installer Minecraft"
  hosts: minecraft_cts
  gather_facts: false
  vars:
    mc_dir: "/opt/minecraft"
    mc_ram: "{{ ram | default('1G') }}"

  tasks:
    - name: Créer le dossier minecraft
      file:
        path: "{{ mc_dir }}"
        state: directory
        mode: "0755"

    - name: Télécharger Minecraft Server
      get_url:
        url: "https://download.oracle.com/java/21/latest/jdk-21_linux-x64_bin.deb"
        dest: /tmp/jdk21.deb

    - name: Installer Java
      apt:
        deb: /tmp/jdk21.deb

    - name: Accepter l'EULA
      copy:
        dest: "{{ mc_dir }}/eula.txt"
        content: "eula=true\n"

    - name: Créer le service systemd
      copy:
        dest: /etc/systemd/system/minecraft.service
        content: |
          [Unit]
          Description=Minecraft Server
          After=network.target

          [Service]
          WorkingDirectory={{ mc_dir }}
          ExecStart=java -Xms512M -Xmx{{ mc_ram }} -jar {{ mc_dir }}/server.jar nogui
          Restart=on-failure

          [Install]
          WantedBy=multi-user.target

    - name: Démarrer Minecraft
      systemd:
        name: minecraft
        daemon_reload: true
        enabled: true
        state: started

    - name: Résumé
      debug:
        msg: "Minecraft pret sur {{ inventory_hostname }}:25565"
```

Hôte cible : groupe minecraft_cts (le conteneur nouvellement créé)

Ce Play s'exécute directement à l'intérieur du conteneur via SSH. Il installe Java 21 et configure le serveur Minecraft comme un service système.

Tâche 1 — Création du répertoire /opt/minecraft

Le module file crée le répertoire de travail du serveur avec les permissions 0755.

Tâche 2 — Téléchargement du JDK Oracle 21

Le module get_url télécharge le paquet .deb du JDK 21 d'Oracle directement sur le conteneur.

```
https://download.oracle.com/java/21/latest/jdk-21_linux-x64_bin.deb
```

Tâche 3 — Installation de Java via apt

Le module apt installe le .deb téléchargé. Le JDK Oracle 21 est utilisé plutôt que le JDK OpenJDK des dépôts Debian, probablement pour des raisons de performances ou de compatibilité avec des versions récentes du serveur Minecraft.

Tâche 4 — Acceptation de l'EULA Minecraft

Minecraft Server refuse de démarrer sans que l'EULA soit explicitement acceptée. Le module copy crée le fichier eula.txt avec la valeur eula=true.

Tâche 5 — Service systemd

Un fichier de service systemd est créé pour que Minecraft démarre automatiquement au boot du conteneur et redémarre en cas de crash (Restart=on-Failure). La RAM allouée au JVM est configurable via la variable mc_ram (défaut : 1G) :

Directive	Valeur
WorkingDirectory	/opt/minecraft
ExecStart	java -Xms512M -Xmx{mc_ram} -jar server.jar nogui
Restart	on-failure
WantedBy	multi-user.target

Tâche 6 — Démarrage du service

Le module systemd recharge le daemon (daemon_reload: true), active le service au boot (enable: true) et le démarre immédiatement (state: started).

5. Flux d'exécution complet

Voici le déroulement chronologique d'une exécution du Playbook depuis la commande ansible-Playbook deploy.yml :

#	Étape	Description
1	Initialisation CSV	Création du fichier ct_registry.csv s'il est absent
2	Comptage	grep compte les lignes existantes dans le CSV
3	Calcul	Détermination du CTID et de l'IP pour le nouveau conteneur
4	Enregistrement	Ajout immédiat de la nouvelle entrée dans le CSV
5	Auth Proxmox	Obtention du ticket de session et du token CSRF
6	Création LXC	Appel API REST POST pour créer le conteneur sur pve-02
7	Démarrage CT	Proxmox démarre le conteneur (paramètre start: 1)
8	Attente SSH	Ansible attend que le port 22 réponde (max 60s)
9	Inventaire dyn.	add_host enregistre le CT dans le groupe minecraft_cts
10	Répertoire	Création de /opt/minecraft dans le CT
11	Download JDK	Téléchargement du JDK Oracle 21 sur le CT
12	Install Java	Installation via apt deb
13	EULA	Création du fichier eula.txt
14	Service	Création du fichier minecraft. Service
15	Start	Activation et démarrage du service systemd
16	Résumé	Affichage de l'IP et du port de connexion

6. Paramètres et variables

Les variables configurables sont définies dans les sections vars des plays respectifs.

Variables du Play 1

Variable	Valeur par défaut	Description
registry_file	./ct_registry.csv	Chemin vers le fichier registre
ip_base	10.2.0	Préfixe des trois premiers octets de l'IP
ip_start	211	Dernier octet de la première IP disponible
id_start	1000	Premier CTID disponible

Variables du Play 2

Variable	Valeur	Description
proxmox_host	10.2.0.253	IP du nœud Proxmox
proxmox_user	root@pam	Utilisateur d'authentification
proxmox_password	root!	Mot de passe (non chiffré)
proxmox_node	pve-02	Nom du nœud Proxmox cible
gateway	10.2.0.254	Passerelle réseau des conteneurs
ssh_key	~/.ssh/id_rsa.pub	Clé publique SSH injectée dans le CT

Variables du Play 3

Variable	Valeur par défaut	Description
mc_dir	/opt/minecraft	Répertoire d'installation du serveur
mc_ram	1G (via --extra-vars)	RAM maximale allouée à la JVM

La variable mc_ram peut être surchargée au lancement :

```
ansible-playbook deploy.yml --extra-vars "ram=2G"
```

7. Points forts et limitations

Points forts

- Déploiement entièrement automatisé en une seule commande
- Attribution automatique d'ID et d'IP uniques via le registre CSV
- Inventaire dynamique : Ansible s'adapte aux nouvelles machines créées
- Isolation des serveurs : chaque instance Minecraft dans son propre LXC
- Service systemd avec redémarrage automatique en cas de crash
- Conteneurs non privilégiés pour une meilleure sécurité
- RAM configurable via variable au moment du déploiement

Limitations identifiées

- Mot de passe Proxmox en clair dans le Playbook (proxmox_password: root!) — risque de sécurité majeur
- Le fichier .jar du serveur Minecraft n'est pas téléchargé : la tâche 'Télécharger Minecraft Server' télécharge en réalité le JDK Oracle, pas le server.jar
- Le registre CSV ne gère pas les suppressions de conteneurs : les IDs et IPs des CT supprimés ne sont pas réutilisés
- Pas de vérification de doublon : si le CSV est corrompu ou modifié manuellement, des conflits d'ID peuvent survenir
- La tâche de démarrage du service systemd cherche un server.jar qui n'a jamais été téléchargé — le service échouera au premier démarrage
- Pas de gestion multi-nœuds : le Playbook est câblé sur pve-02 uniquement
- Pas de pare-feu configuré sur les conteneurs

8. Recommandations d'amélioration

Sécurité

- Stocker le mot de passe Proxmox dans un Ansible Vault chiffré (ansible-vault create secrets.yml) plutôt qu'en clair dans le Playbook
- Utiliser un utilisateur Proxmox dédié avec des permissions limitées (création LXC uniquement) plutôt que root@pam
- Activer un pare-feu basique (ufw ou iptables) dans chaque conteneur pour n'exposer que le port 25565

Fiabilité

- Corriger la tâche de téléchargement Minecraft : ajouter une tâche get_url pour récupérer le server.jar depuis l'API officielle PaperMC ou Mojang
- Ajouter une vérification d'unicité avant l'écriture dans le CSV (vérifier que le CTID n'existe pas déjà dans Proxmox via API)
- Implémenter une tâche de rollback qui supprime l'entrée CSV si la création du conteneur échoue

Maintenabilité

- Externaliser toutes les variables sensibles et configurables dans un fichier group vars/all.yml
- Ajouter des tags Ansible (--tags create, --tags install) pour pouvoir rejouer uniquement une partie du Playbook
- Implémenter un Play de suppression de conteneur (destroy.yml) qui nettoie aussi le registre CSV
- Ajouter une tâche de vérification finale : tester que le port 25565 répond bien avant d'afficher le résumé